DATA STRUCTURES

ETCCDS202

Assignment-3

Topic: Sorting Performance Analyzer (Unit-3)

Short Analysis Report

Submitted By: Meghna Kumar
Roll No.: 2501730214
Program: B.Tech CSE (AI & ML)
Section: A
Faculty Name: Ms Neetu Chauhan
Submitted on: 17-04-2026

Assignment Overview:

The assignment entails creating an SPA program written in Python to test and compare the three sorting techniques (Insertion sort, Merge sort, and Quick sort). Various data types are used by the program to determine their respective processing times. The findings will help us analyze the relationship between real-world performance and theoretical complexities.

Table of recorded execution times:

The following table presents the recorded execution times (in milliseconds) for Insertion Sort, Merge Sort, and Quick Sort across different dataset sizes and input types:

Size	Input Type	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
1000	Random	214.5264	8.1356	4.5159
1000	Sorted	0.5535	18.8835	275.8659
1000	Reverse	260.2385	5.4329	245.0679
5000	Random	2839.7427	69.3477	25.2010
5000	Sorted	2.5000	32.6211	2865.1898
5000	Reverse	3703.0002	25.0387	1802.2233
10000	Random	6005.1635	62.1501	19.2238
10000	Sorted	3.1620	34.4291	10400.1498
10000	Reverse	11777.9232	22.7878	6673.6615

The values in the table were obtained by executing each sorting algorithm on the same datasets and measuring their execution time using a consistent timing method. This ensures a fair and accurate comparison of their performance under different input conditions.

A short comparison paragraph for each input type:

- **Random:** If we take random inputs, Merge Sort and Quick Sort give better performance compared to Insertion Sort. This is due to the reason that both have time complexity as O(n log n). However, in case of Insertion Sort, the time complexity will be

$O(n^2)$, which is relatively higher than other two sorting methods.

- **Sorted:** The best sort for a sorted list is the Insertion Sort algorithm, because its time complexity is O(n). The performance of the Merge Sort algorithm remains stable irrespective of the sequence of the numbers entered, while the worst scenario occurs in Quick Sort if we select a poor pivot position, i.e., the last one.
- **Reverse Sorted:** Insertion Sort has its worst performance for reverse sorted input since it requires the most number of shifts for elements, making the time complexity $O(n^2)$. The performance of Quick Sort is poor due to imbalance during partitioning, but Merge Sort still maintains efficiency with O(n log n) time.

<u>Discussion: How does $O(n^2)$ vs O(nlogn) match your timing result?</u>

It is evident from the experimental results that the performance difference between $O(n^2)$ and O(n log n) complexities has been captured effectively. While Insertion Sort algorithm, whose complexity is $O(n^2)$, increases rapidly with increasing values of n in case of random as well as reverse sorted input data, Merge Sort and Quick Sort, whose average complexity is O(n log n), perform better with increasing values of n and handle large data sets very effectively.

<u>Special note for Quick Sort:</u>

- **<u>Mention worst-case O(n 2) when pivot is poor (e.g., sorted input with last element pivot).</u>**
  The average complexity of Quick Sort is O(n log n), whereas its worst-case complexity is $O(n^2)$. The reason behind the worst-case complexity is the poor choice of the pivot value. In this code, the last element is selected as the pivot value. If the array

values are already sorted in ascending or descending order, the choice of the pivot element causes the partitioning to be highly unbalanced. One subarray will have almost all elements, while the other subarray will have few or no elements.

- **<u>If you observe slow performance for sorted or reverse-sorted, explain why.</u>**
  The reason why Quick Sort operates slowly when applied to an already sorted or reverse sorted data sequence is improper choice of pivot. In case the last element serves as the pivot, sorted or reverse sorted arrays lead to extremely skewed partitions – either one part will contain almost all elements and the other will be empty. This leads to increased recursion with the processing of almost the whole array at each recursion step. The amount of comparisons made becomes considerably higher, which makes Quick Sort operate slower.

<u>Stability and memory notes:</u>

- **<u>Insertion sort: stable, in-place</u>**
  Insertion Sort is an example of a stable as well as an in-place sorting technique. Stability refers to the fact that if two numbers are equal, then they would maintain their order with respect to each other even after sorting. An in-place sort is a sort where no extra space is used, and sorting takes place on the array itself.
- **<u>Merge sort: stable, out-of-place (uses extra space)</u>**
  Merge Sort is another example of a stable sorting algorithm because it preserves the relative order of duplicate values. But Merge Sort differs from Insertion Sort in that it is not an in-place algorithm. Instead, it takes more memory to perform its operations because it utilizes temporary arrays when merging the divided lists. Still, it is memory-intensive and works consistently with a time complexity of O(n log n).
- **<u>Quick sort: usually in-place, not stable (in typical implementation)</u>**

Quick Sort can be categorized as an in-place sort since it uses partitioning to sort the array by shifting around elements within the same list. Nevertheless, Quick Sort cannot be referred to as stable because when the sort occurs, equal elements can swap places. Despite its efficiency, it would be prudent to take into consideration its weaknesses before opting to use it.

Conclusion:

To summarize, the assignment highlighted the practical difference among several sorting algorithms regarding their efficiency, stability, and memory consumption. It turned out that sorting algorithms of O(n log n) complexity such as Merge Sort and Quick Sort performed much better than $O(n^2)$ Insertion Sort when processing large amounts of information. On the other hand, the effectiveness of Quick Sort highly depends on the pivot choice and is evident by its inferior performance when working with sorted and reversed sequences of numbers. At the same time, Merge Sort exhibited stable performance regardless of the type of input, whereas Insertion Sort was efficient in processing small arrays or sorted elements only.